

TECHNICAL REPORT

Secure Private Storage Server

Secure NAS

Course Unit	Operating Systems
Project	Secure NAS – Phase 1 of Final Project
Platform	VirtualBox
Technical Scope	LVM - DAC - MAC (AppArmor) - Firewall / IDS - Backups
Operating System	Ubuntu Server 24.04 LTS
Author	Leandro Fernandes

This report documents the design and hardened configuration of a private Network Attached Storage (NAS) server on Linux. It covers hardware abstraction through LVM, Discretionary Access Control (DAC), kernel-level process confinement via Mandatory Access Control (AppArmor), perimeter security with the UFW firewall and Fail2Ban intrusion detection, and resilience automation with Systemd timers. The dual network-interface architecture prepares the system to evolve into a gateway in the second phase of the project, in line with the Defense-in-Depth model.

LVM Storage	DAC Identities	MAC AppArmor	UFW Firewall	Fail2Ban IDS	Systemd Backups
----------------	-------------------	-----------------	-----------------	-----------------	--------------------

Content

Project Objectives	3
1. Hardware Architecture & Base OS	5
1.1 System Specifications.....	6
2. Storage Abstraction (LVM)	7
2.1 Building the LVM Stack.....	8
3. Identity & Permission Management (DAC)	9
3.1 Identity & Group Structure	9
3.2 Permissions Map	10
4. Sharing & Isolation Services (Samba & SFTP).....	11
4.1 Samba Server (SMB / CIFS).....	11
4.2 Isolated SFTP Server (chroot)	12
5. Advanced Security (AppArmor / MAC)	15
5.1 AppArmor -9 Local Override (Upgrade-Safe).....	15
5.2 AppArmor Operating Modes	16
6. Perimeter Security (Firewall & IDS).....	17
6.1 UFW Firewall - Per-Interface Policy	17
6.2 Fail2Ban - Brute-Force Mitigation	17
7. Backup Automation (Systemd Timers)	19
7.1 Backup Script	19
7.2 Systemd Units	19
8. Validation & Conclusion	22
8.1 Functional Validation Procedure	22
8.2 Identity, Service & Resource Map.....	22
8.3 Threat & Mitigation Matrix	23
8.4 Conclusion.....	23
Appendix A — Acronym Glossary	24

Project Objectives

The Secure NAS project delivers a private, secure and resilient storage platform. The six objectives below map directly onto the layered Defense-in-Depth model shown in Figure 1 - each layer is an independent control that continues to protect the data even if another layer is bypassed.

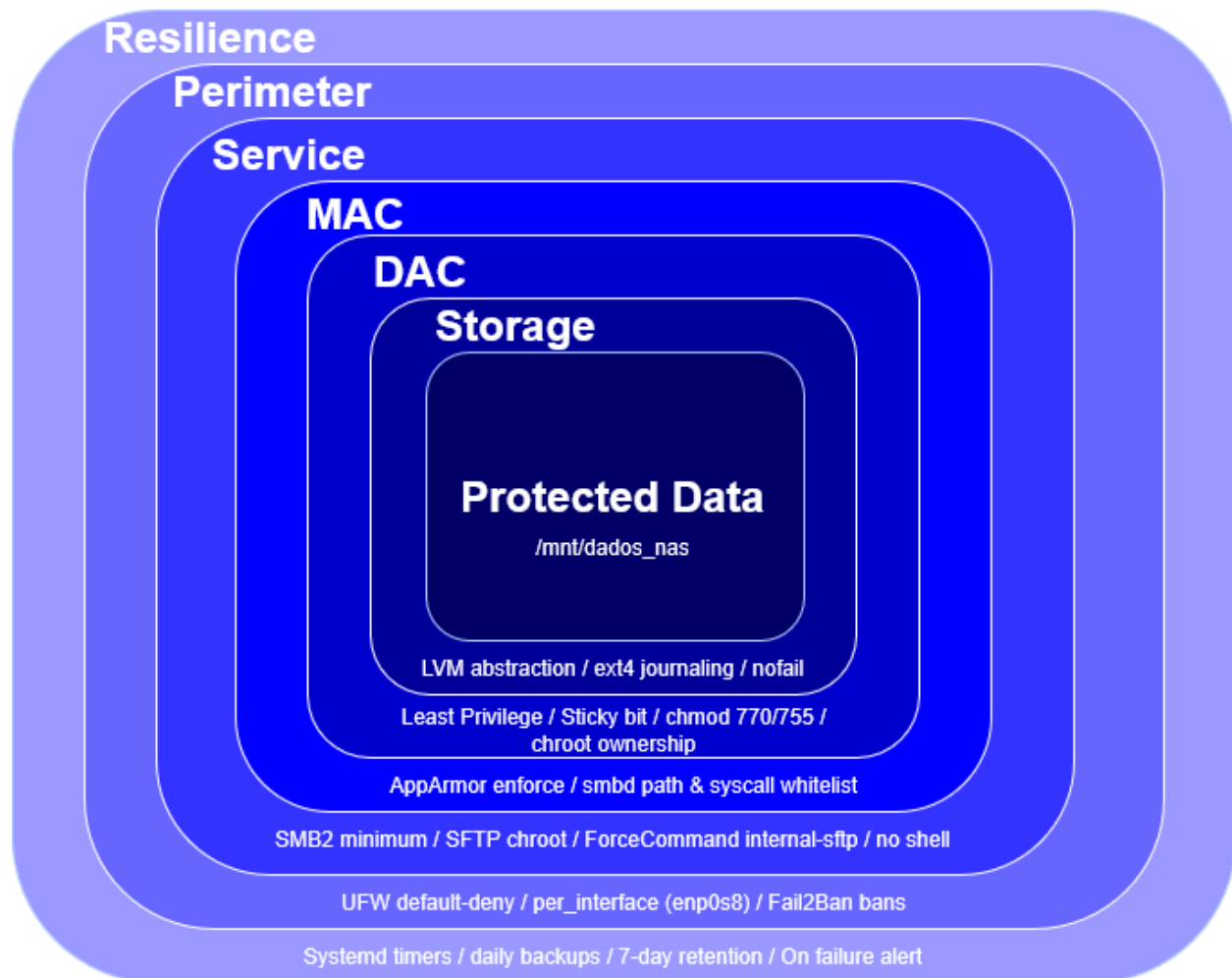


Figure 1 - Defense-in-Depth: six independent layers wrapping the protected data.

LVM - Storage Abstraction

Abstract physical storage through LVM, enabling hot expansion with no downtime and dynamic capacity optimization.

DAC - Least Privilege

Segment identities and privileges under the Principle of Least Privilege, so each user reaches only the resources strictly required.

AppArmor / MAC — Process Confinement

Confine service processes at the kernel level with Mandatory Access Control, containing remote code execution (RCE).

Samba / SFTP - Secure Interoperability

Provide interoperable shares (Samba / SMB) and isolated, encrypted transfers (SFTP in chroot) with no interactive shell.

UFW + Fail2Ban — Attack-Surface Reduction

Shrink the attack surface with a per-interface firewall and automatic brute-force mitigation.

Systemd Timers — Automated Resilience

Guarantee resilience through automatic backups with configurable retention, failure alerting and full configuration recovery.

1. Hardware Architecture & Base OS

Separating the operating-system disk (sda) from the data disks (sdb, sdc) ensures that a catastrophic system failure — or storage exhaustion caused by users — neither corrupts nor halts the core infrastructure. The dual network interface isolates management / internet traffic (NAT) from local traffic (Host-only), preparing the server to act as a gateway in Phase 2.

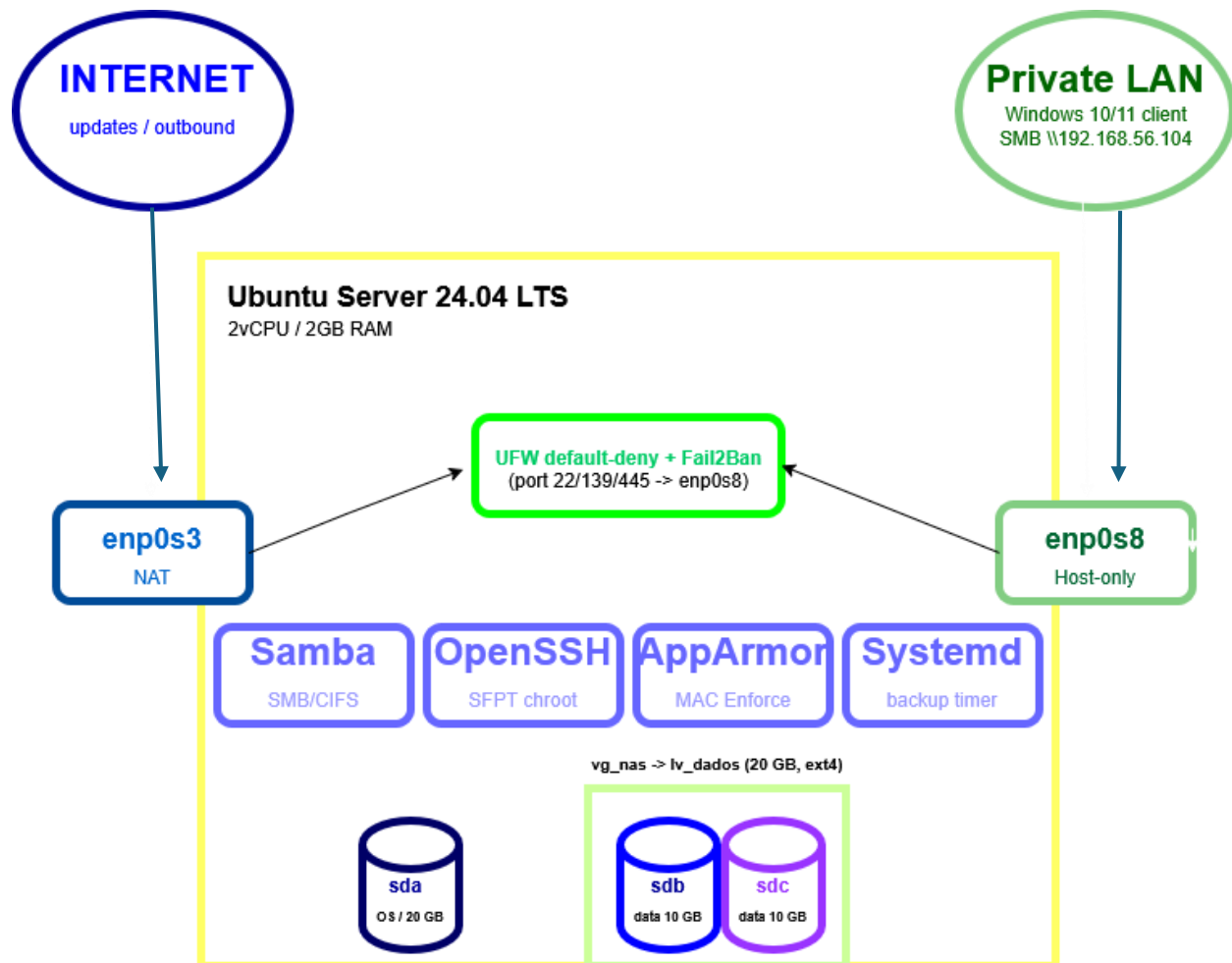


Figure 2 - System and network topology: two isolated NICs, three disks, and the confined services.

1.1 System Specifications

Component	Technical / Operational Specification
Operating System	Ubuntu Server 24.04 LTS
Virtual Hardware	2 vCPUs (x86_64), 2048 MB RAM — VirtualBox
enp0s3 interface	NAT — external / internet access (outbound traffic and updates)
enp0s8 interface	Host-only — private local network (NAS services and management)
sda disk (20 GB)	Operating System — boot, root and swap partitions
sdb disk (10 GB)	Data — Physical Volume for Volume Group vg_nas
sdc disk (10 GB)	Data — Physical Volume for Volume Group vg_nas

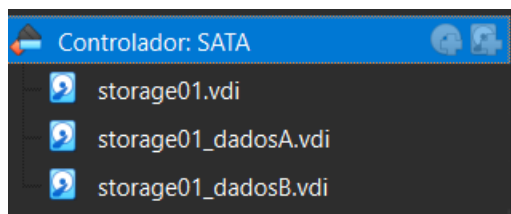


Figure 3 – Hypervisor storage / the three disks (sda, sdb, sdc)

```

user@storage01:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:7c:af:fa brd ff:ff:ff:ff:ff:ff
    altname enx0800277c affa
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86372sec preferred_lft 86372sec
    inet6 fd17:625c:f037:2:a00:27ff:fe7c:affa/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86373sec preferred_lft 14373sec
    inet6 fe80::a00:27ff:fe7c:affa/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:d0:33:15 brd ff:ff:ff:ff:ff:ff
    altname enx080027d0 3315
    inet 192.168.56.104/24 metric 1024 brd 192.168.56.255 scope global dynamic enp0s8
        valid_lft 572sec preferred_lft 572sec
    inet6 fe80::a00:27ff:fed0:3315/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
  
```

Figure 4 – Hypervisor network setting / the two active network adapters

2. Storage Abstraction (LVM)

The Logical Volume Manager (LVM) provides an abstraction layer over physical hardware, letting independent disks be aggregated and storage be expanded hot, with no downtime. The two 10 GB disks (`sdb` and `sdc`) merge into a single 20 GB Volume Group (`vg_nas`), optimizing provisioning and preparing the infrastructure for future scaling.

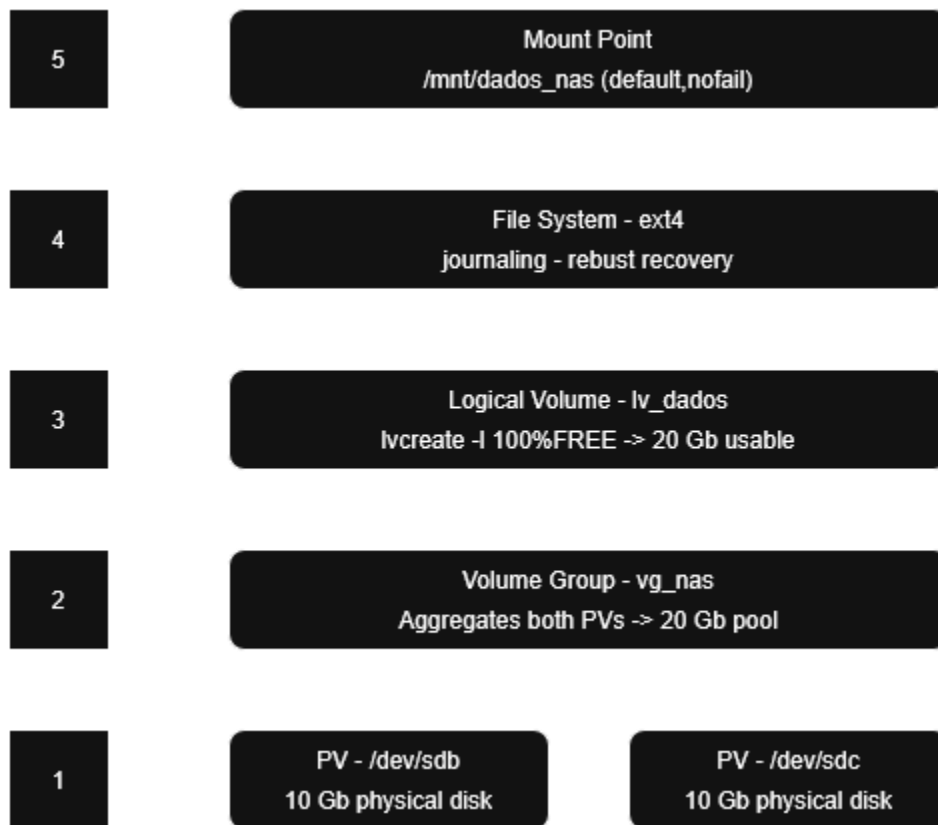


Figure 5 - The LVM stack, built bottom-up from two physical volumes to the mounted filesystem.

2.1 Building the LVM Stack

Configuration commands

```
# 1. Initialize the physical disks as Physical Volumes
sudo pvcreate /dev/sdb /dev/sdc

# 2. Create the Volume Group aggregating both Physical Volumes
sudo vgcreate vg_nas /dev/sdb /dev/sdc

# 3. Create the Logical Volume consuming 100% of available space
sudo lvcreate -n lv_dados -l 100%FREE vg_nas

# 4. Format with ext4 (maturity, journaling, robust recovery)
sudo mkfs.ext4 /dev/vg_nas/lv_dados

# 5. Create the mount point and mount the volume
sudo mkdir -p /mnt/dados_nas
sudo mount /dev/vg_nas/lv_dados /mnt/dados_nas
```

Boot persistence – sudo nano /etc/fstab

```
/dev/vg_nas/lv_dados    /mnt/dados_nas    ext4    defaults,nofail    0 0
```

```
User@storage01:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda          8:0    0   20G  0 disk
├─sda1       8:1    0    1M  0 part
└─sda2       8:2    0   20G  0 part /
sdb          8:16   0    10G  0 disk
└─vg_nas-lv_dados 252:0   0   20G  0 lvm  /mnt/dados_nas
sdc          8:32   0    10G  0 disk
└─vg_nas-lv_dados 252:0   0   20G  0 lvm  /mnt/dados_nas
sr0         11:0    1 1024M  0 rom
```

Figure 6 – Hierarchy tree: sdb and sdc mapped to vg_nas-lv_dados

```
User@storage01:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            329M  4.1M  325M   2% /run
/dev/sda2       20G   3.1G   16G  17% /
tmpfs            822M    0  822M   0% /dev/shm
tmpfs            822M    0  822M   0% /tmp
none            1.0M    0   1.0M   0% /run/credentials/systemd-journald.service
none            1.0M    0   1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M    0   1.0M   0% /run/credentials/systemd-networkd.service
none            1.0M    0   1.0M   0% /run/credentials/getty@tty1.service
/dev/mapper/vg_nas-lv_dados 20G   2.1M   19G   1% /mnt/dados_nas
tmpfs           165M    0  165M   1% /run/user/1000
```

Figure 7 – Output of “df -h” – We can see 20 GB mounted at /mnt/dados_nas

3. Identity & Permission Management (DAC)

Discretionary Access Control (DAC) realizes the Principle of Least Privilege. OpenSSH enforces strict root-ownership rules to enable chroot isolation, while the Sticky Bit on the public directory prevents malicious or accidental deletion of other users' files even where write permission exists (chmod 777).

The /mnt/dados_nas/sftp_jail structure must satisfy OpenSSH's strict constraints: the chroot parent directory must be owned by root:root and must NOT be writable by group or others (chmod 755). Non-compliance makes the SSH daemon refuse the connection immediately with "bad ownership or modes."

3.1 Identity & Group Structure

```
# 1. Create logical security groups
sudo groupadd sftpusers
sudo groupadd smb_privado

# 2. Create users and assign them to groups
sudo adduser user_samba
sudo usermod -aG smb_privado user_samba
sudo adduser user_sftp
sudo usermod -aG sftpusers user_sftp

# 3. Create the directory structure on the NAS volume
sudo mkdir -p /mnt/dados_nas/publico
sudo mkdir -p /mnt/dados_nas/privado
sudo mkdir -p /mnt/dados_nas/sftp_jail/uploads

# 4. Public folder: full access + Sticky Bit (prevents cross-deletion)
sudo chmod 777 /mnt/dados_nas/publico
sudo chmod +t /mnt/dados_nas/publico      # Resulting mask: drwxrwxrwt

# 5. Private folder: exclusive to the smb_privado group
sudo chown root:smb_privado /mnt/dados_nas/privado
sudo chmod 770 /mnt/dados_nas/privado

# 6. SFTP chroot: root ownership, 755 (OpenSSH requirement)
sudo chown root:root /mnt/dados_nas/sftp_jail
sudo chmod 755 /mnt/dados_nas/sftp_jail
sudo chown user_sftp:sftpusers /mnt/dados_nas/sftp_jail/uploads
```

3.2 Permissions Map

Directory	Owner:Group	Permission	Purpose
/mnt/dados_nas/publico	root:root	1777	Public share — Sticky Bit prevents cross-deletion
/mnt/dados_nas/privado	root:smb_privado	0770	Exclusive access for the smb_privado group
/mnt/dados_nas/sftp_jail	root:root	0755	chroot root — ownership mandated by OpenSSH
.../sftp_jail/uploads	user_sftp:sftpusers	0750	Writable directory inside the SFTP jail

```
user@storage01:~$ ls -ld /mnt/dados_nas/publico
drwxrwxrwt 2 root root 4096 Jun 25 18:26 /mnt/dados_nas/publico
```

Figure 8 – Output of “ls -ld /mnt/dados_nas/publico” – We can see 20 GB mounted at /mnt/dados_nas – The sticky bit shows the t marker in the permission mask (drwxrwxrwt)

4. Sharing & Isolation Services (Samba & SFTP)

Samba exposes the SMB/CIFS protocol, giving native interoperability with Windows clients without extra software. SFTP (over OpenSSH) provides encrypted remote transfers with key-pair authentication. The ChrootDirectory directive confines SFTP accounts to an isolated space with no visibility of the root filesystem and no interactive shell, mitigating privilege-escalation vectors and unauthorized reading of system files.

4.1 Samba Server (SMB / CIFS)

User registration and validation

```
# Register the user in the Samba password database
sudo smbpasswd -a user_samba

# Validate smb.conf syntax before applying (no downtime)
sudo testparm
```

File: /etc/samba/smb.conf

```
[global]
    map to guest = bad user          # Map unauthenticated requests to guest
    server min protocol = SMB2      # Disable SMB1 (EternalBlue / CVE-2017-0144)

[Publico]
    path = /mnt/dados_nas/publico
    browseable = yes
    writable = yes
    guest ok = yes
    public = yes
    force create mode = 0777
    force directory mode = 0777

[Privado]
    path = /mnt/dados_nas/privado
    browseable = yes
    writable = yes
    guest ok = no
    valid users = @smb_privado      # smb_privado group members only
    create mask = 0660              # New files:  rw-rw----
    directory mask = 0770           # New folders: rwxrwx---
```

The *map to guest = bad user* directive resolves anonymous access to the public share for modern clients (Windows 10/11), mapping requests with invalid credentials to the system guest account instead of returning an authentication error that would block access.

The *server min protocol = SMB2* directive fully disables SMB version 1, removing the attack surface exploited by EternalBlue (CVE-2017-0144; WannaCry / NotPetya ransomware). The *create mask = 0660* and *directory mask = 0770* masks enable collaborative editing among group members, preventing files created by one member from becoming inaccessible to the others.

4.2 Isolated SFTP Server (chroot)

File: /etc/ssh/sshd_config — append at the end

```
Match Group sftpusers
    ChrootDirectory /mnt/dados_nas/sftp_jail    # User's apparent root
    ForceCommand internal-sftp                 # No interactive shell
    AllowTcpForwarding no                      # Block TCP tunnels
    X11Forwarding no                           # Block graphical forwarding
```

Validation and activation

```
# Validate syntax before restarting (prevents lockout on a config error)
sudo testparm                # Validates smb.conf
sudo sshd -t                 # Validates sshd_config – errors surface first

# Apply the configuration
sudo systemctl restart smbd
sudo systemctl restart ssh
```

“Remote access” here means access from the private local network (interface `enp0s8`). As detailed in Section 6, the UFW firewall restricts the SSH/SFTP (22) and Samba (139/445) ports exclusively to the internal interface, never exposing these services on the internet-facing NAT interface (`enp0s3`).

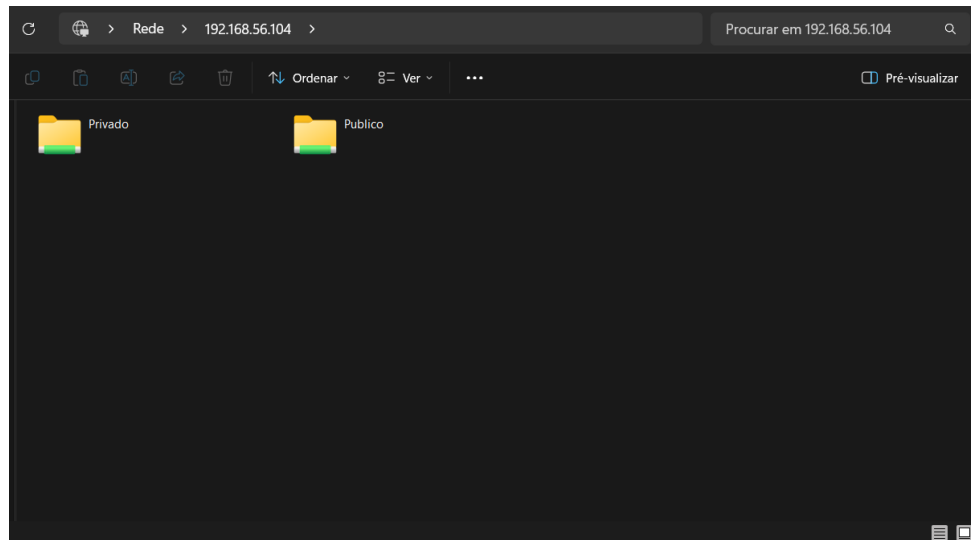


Figure 9 – Windows File Explorer with the shares mapped via UNC (\\192.168.56.104)

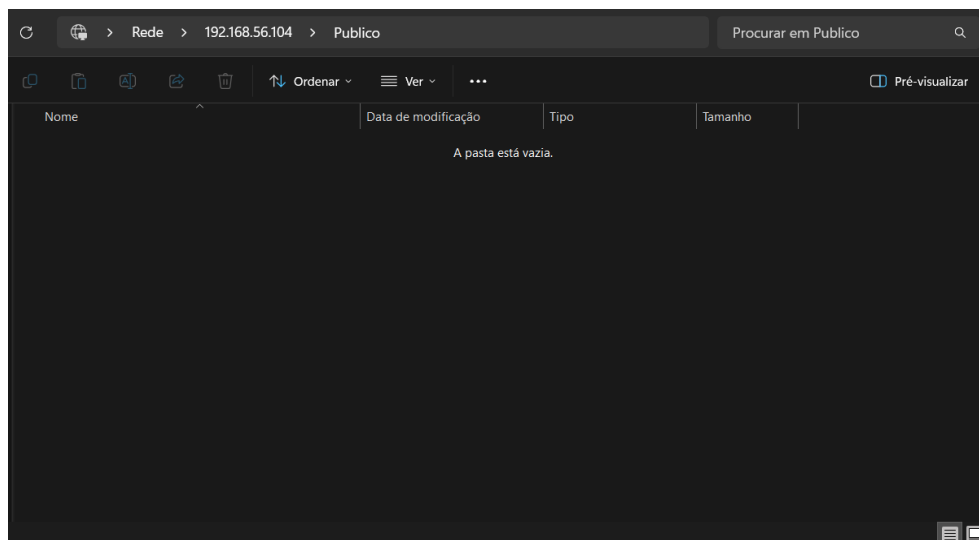


Figure 10 – Validated access to the Publico folder

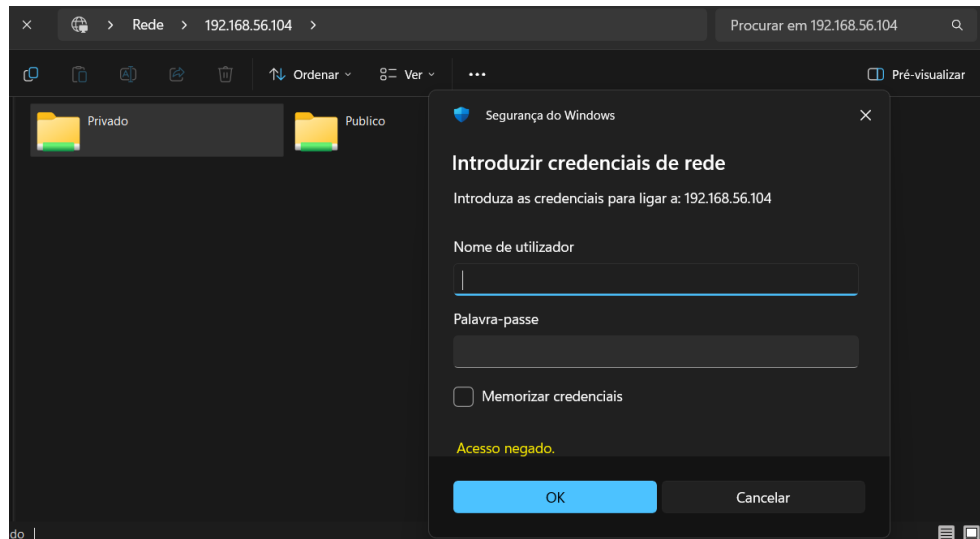


Figure 11 – Validated access to the Privado folder

5. Advanced Security (AppArmor / MAC)

Mandatory Access Control (MAC), implemented by AppArmor at the Linux kernel level, isolates the operational privileges of the Samba process itself. If the `smbd` daemon is exploited through zero-day vulnerability or remote code execution (RCE), the AppArmor rules restrict the available syscalls, blocking access to any path not declared in the whitelist and safeguarding the integrity of the operating system.

5.1 AppArmor -9 Local Override (Upgrade-Safe)

Rather than editing the main profile shipped by the package (which would be overwritten on updates), LVM access permission is injected into the local override file. This keeps the configuration persistent across software updates (`apt upgrade`).

```
# 1. Install AppArmor utilities and base profiles
sudo apt install apparmor-utils
sudo apt install apparmor-profiles

# 2. Inject the permission into the Samba profile's local override
#     (survives smbd package updates)
echo '/mnt/dados_nas/** rwk,' | sudo tee -a /etc/apparmor.d/local/usr.sbin.smbd

# 3. Reload the profile into the kernel with the new permissions
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.smbd

# 4. Force Enforce mode (from Complain to Enforce)
sudo aa-enforce /usr/sbin/smbd

# 5. Restart Samba to apply the updated profile
sudo systemctl restart smbd
```

5.2 AppArmor Operating Modes

AppArmor Mode	Behavior	Recommended Use
Complain (Audit)	Logs violations to the system log but does not block them	Profile testing and refinement
Enforce (Production)	Actively blocks any action not in the whitelist	Mandatory in production
Disabled	Profile inactive - no active MAC protection	Not recommended in production

```
user@storage01:~$ sudo aa-status | grep smbd
smbd
/usr/sbin/smbd (1392) smbd
/usr/sbin/smbd (1403) smbd
/usr/sbin/smbd (1404) smbd
/usr/sbin/smbd (1949) smbd
```

Figure 12 – Output of “sudo aa-status | grep smbd”

6. Perimeter Security (Firewall & IDS)

Attack-surface reduction is achieved with a firewall enforcing a default-deny policy, opening services exclusively on the internal interface (enp0s8). As an additional layer, Fail2Ban monitors authentication attempts through the systemd journal and automatically blocks offending IP addresses, turning brute-force attacks into auditable, neutralized events.

6.1 UFW Firewall - Per-Interface Policy

```
# 1. Base policy: deny all inbound traffic by default
sudo ufw default deny incoming
sudo ufw default allow outgoing

# 2. Open services ONLY on the internal interface (private LAN)
sudo ufw allow in on enp0s8 to any port 22 proto tcp    # SSH / SFTP
sudo ufw allow in on enp0s8 to any port 139 proto tcp   # Samba – NetBIOS
sudo ufw allow in on enp0s8 to any port 445 proto tcp   # Samba – Direct SMB

# 3. Enable the firewall
sudo ufw enable

# 4. Verify the active rules
sudo ufw status verbose
```

6.2 Fail2Ban - Brute-Force Mitigation

```
sudo apt install fail2ban
sudo systemctl enable --now fail2ban
```

File: `/etc/fail2ban/jail.local`

```
[DEFAULT]
# CRITICAL on Ubuntu Server 24.04: rsyslog is not installed by default,
# so /var/log/auth.log may not exist. The 'systemd' backend reads the
# systemd-journald journal directly, ensuring effective detection.
backend = systemd
bantime  = 1h          # Automatic ban duration (1 hour)
findtime = 10m         # Window for counting attempts
maxretry = 5          # Failures allowed before a ban

[sshd]
enabled = true
port    = 22
```

Ubuntu Server 24.04 LTS dropped rsyslog from the default installation. Without the *backend = systemd* directive, Fail2Ban tries to read */var/log/auth.log*, which may not exist, leaving SSH completely unprotected even though the service appears active. Setting *backend = systemd* is mandatory.

```
user@storage01:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
139,445/tcp on enp0s8 ALLOW IN Anywhere
22/tcp on enp0s8 ALLOW IN Anywhere
139,445/tcp (v6) on enp0s8 ALLOW IN Anywhere (v6)
22/tcp (v6) on enp0s8 ALLOW IN Anywhere (v6)
```

Figure 13 – Output of “*sudo ufw status verbose*” - Per-interface rules, deny incoming

```
user@storage01:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 0
|   |- Total failed: 0
|   \- Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
\-- Actions
    |- Currently banned: 0
    |- Total banned: 0
    \- Banned IP list:
```

Figure 14 – Output of “*sudo fail2ban-client status sshd*” - Active jail, banned-IP counter

7. Backup Automation (Systemd Timers)

Systemd timers are the native, modern approach to scheduling on Linux, with centralized logging via journalctl, strict control over mount dependencies, and integration with the unit system. The `Requires` directive aborts the job if the NAS volume is not mounted, preventing silent logical failures. `OnFailure` turns a passive log entry into an active notification.

7.1 Backup Script

File: `/usr/local/bin/nas_backup.sh`

```
#!/bin/bash
# Automated NAS Backup Script – Secure NAS Phase 1
DATA=$(date +%Y%m%d_%H%M%S)
DESTINO="/mnt/dados_nas/backups/backup_${DATA}.tar.gz"

# Ensure the destination directory exists beforehand
# (avoids 'No such file or directory' on the first run)
mkdir -p /mnt/dados_nas/backups

# Archive data + full infrastructure configuration
# Errors are visible in: journalctl -u nas-backup.service
tar -czf "$DESTINO" \
    /mnt/dados_nas/publico \
    /mnt/dados_nas/privado \
    /etc/samba/smb.conf \
    /etc/ssh/sshd_config \
    /etc/systemd/system/nas-backup.*

# Retention policy: remove backups older than 7 days
find /mnt/dados_nas/backups/ -type f -mtime +7 -delete

# After that, save the file and exit to the command prompt and make the script executable
sudo chmod +x /usr/local/bin/nas_backup.sh
```

7.2 Systemd Units

File: `/etc/systemd/system/nas-backup.service`

```
[Unit]
Description=NAS Shares Backup Service
Requires=mnt-dados_nas.mount # Aborted if the volume is not mounted
After=mnt-dados_nas.mount
OnFailure=nas-backup-alert.service # Active alert on error

[Service]
Type=oneshot
ExecStart=/usr/local/bin/nas_backup.sh
```

File: /etc/systemd/system/nas-backup-alert.service

```
[Unit]
Description=NAS Backup Failure Alert

[Service]
Type=oneshot
ExecStart=/usr/bin/logger -p user.err \
    "CRITICAL ALERT: NAS backup failed! Check: journalctl -u nas-backup.service"
```

File: /etc/systemd/system/nas-backup.timer

```
[Unit]
Description=Daily Timer for Automated NAS Backup

[Timer]
OnCalendar=*-*-* 02:00:00    # Runs daily at 02:00 (low load)
Persistent=true             # Recovers missed runs after downtime

[Install]
WantedBy=timers.target
```

Final activation

```
# Register the new units with systemd
sudo systemctl daemon-reload

# Enable and start the timer (persistent across reboots)
sudo systemctl enable --now nas-backup.timer

# Verify the schedule
systemctl list-timers nas-backup.timer
```

Retention: *find -mtime +7 -delete* prevents Storage-Full failures (keeps 7 days of history).

Alerting: OnFailure emits a message to the system journal — errors are never silent.

Disaster Recovery: the archive includes data (publico, privado) and the full infrastructure configuration (smb.conf, sshd_config, systemd units), enabling a complete rebuild.

The backup currently resides in the same Volume Group as the data (a partial violation of the 3-2-1 rule). In future iterations the destination should be externalized to an independent volume or a remote target. Consistency for heavily used files can be guaranteed with an LVM Snapshot taken before compression.

```
user@storage01:~$ ls -lh /mnt/dados_nas/backups/
total 12K
-rw-r--r-- 1 root root 165 Jun 25 19:43 backup_20260625_194312.tar.gz
-rw-r--r-- 1 root root 5.6K Jun 26 12:09 backup_20260626_120900.tar.gz
```

Figure 15 – Output of “ls -lh /mnt/dados_nas/backups/” – A “backup_YYYYMMDD_HHMMSS.tar.gz” archive created with the correct timestamp and a non-zero size.

8. Validation & Conclusion

8.1 Functional Validation Procedure

Each control was validated actively and reproducibly. The table maps control → command → expected result, turning the configuration into an auditable verification.

Control	§	Verification Command	Expected Result
LVM stack	§2	<code>lsblk; df -h /mnt/dados_nas</code>	<code>vg_nas-lv_dados</code> (~20 GB) mounted
Sticky Bit	§3	<code>ls -ld /mnt/dados_nas/publico</code>	Permissions <code>drwxrwxrwt</code>
SFTP chroot	§4	<code>sftp user_sftp@<IP> → cd /etc</code>	Access denied outside jail; no shell
Private share	§4	<code>smbclient //<IP>/Privado -U <ext></code>	<code>NT_STATUS_ACCESS_DENIED</code>
SMB1 disabled	§4	<code>testparm -v grep 'min protocol'</code>	<code>server min protocol = SMB2</code>
AppArmor / MAC	§5	<code>sudo aa-status grep smbd</code>	<code>smbd</code> listed in enforce mode
UFW firewall	§6	<code>sudo ufw status verbose</code>	deny incoming; 22/139/445 on <code>enp0s8</code>
Fail2Ban / IDS	§6	<code>sudo fail2ban-client status sshd</code>	Active jail; banned-IP counter
Manual backup	§7	<code>sudo systemctl start nas-backup</code>	<code>.tar.gz</code> created with correct timestamp
Daily timer	§7	<code>systemctl list-timers nas-backup</code>	Next run at 02:00 (NEXT field)

8.2 Identity, Service & Resource Map

User	Group	Service	Share / Path	Perms
(guest)	—	Samba (SMB)	<code>/mnt/dados_nas/publico</code>	<code>1777</code>
<code>user_samba</code>	<code>smb_privado</code>	Samba (SMB)	<code>/mnt/dados_nas/privado</code>	<code>0770</code>
<code>user_sftp</code>	<code>sftpusers</code>	SFTP (chroot)	<code>.../sftp_jail/uploads</code>	<code>0750</code>

8.3 Threat & Mitigation Matrix

Threat / Risk	Implemented Control	Layer
SMB1 / EternalBlue (CVE-2017-0144)	<code>server min protocol = SMB2</code>	Service (§4)
SSH brute force	Fail2Ban — ban after 5 fails / 10 min	Perimeter (§6)
Services exposed to the internet	UFW default-deny + per-interface (enp0s8)	Perimeter (§6)
RCE / zero-day in smbd	AppArmor enforce — path & syscall whitelist	MAC (§5)
SFTP escape to the system	ChrootDirectory + <code>internal-sftp</code>	Isolation (§3–4)
Malicious deletion of public files	Sticky Bit on the public folder (1777)	DAC (§3)
Storage exhaustion from backups	<code>find -mtime +7 -delete</code> (7 days)	Resilience (§7)
Volume failure at OS boot	<code>nofail</code> in fstab — no Emergency Mode	Storage (§2)
Loss of data and configuration	Daily backup (data + config) + OnFailure	Resilience (§7)

8.4 Conclusion

LVM Abstracted, hot-expandable storage (sdb + sdc → 20 GB vg_nas).

DAC Access segregated by identity and group (Sticky Bit, chroot, 770/755).

MAC Service processes confined at the kernel level (AppArmor enforce).

UFW Attack surface reduced by a default-deny, per-interface firewall.

Fail2Ban Brute force automatically mitigated with per-IP bans (systemd backend).

Systemd Resilience via automatic backups with retention, alerting and disaster recovery.

The dual-interface architecture (NAT + Host-only) leaves the system ready for Phase 2, where it is expected to operate as a gateway and externalize backups under the 3-2-1 rule (3 copies, 2 distinct media, 1 offsite).

Appendix A — Acronym Glossary

A quick reference to the technical terminology used throughout this report.

Acronym	Term	Role in the Project
LVM	Logical Volume Manager	Abstracts physical disks into a hot-expandable logical volume
PV / VG / LV	Physical Volume / Volume Group / Logical Volume	Layers of the LVM storage stack
DAC	Discretionary Access Control	User/group permissions under least privilege
MAC	Mandatory Access Control	Kernel-enforced process confinement via AppArmor
SMB / CIFS	Server Message Block / Common Internet File System	File-sharing protocol served by Samba
SFTP	SSH File Transfer Protocol	Encrypted file transfer over OpenSSH
chroot	Change Root	Confines a process to an isolated filesystem subtree
UFW	Uncomplicated Firewall	Management frontend for the Ubuntu iptables firewall
IDS	Intrusion Detection System	Detection and blocking — implemented via Fail2Ban
RCE	Remote Code Execution	Remote code execution — contained by MAC confinement
LTS	Long Term Support	Ubuntu release with extended support (24.04 until 2029)
NAT	Network Address Translation	IP translation — used on the enp0s3 interface
CVE	Common Vulnerabilities and Exposures	Public identification system for security vulnerabilities